



利用跨 GPU 共享内存实现无缝的 MoE 通信-计算融合

Hulin Wang
wonghulin@whu.edu.cn 计
算机学院 武汉大学 武汉，中
国

Yaqi Xia
yaqixia@whu.edu.cn
武汉大学计算机学院，中
国武汉

Donglin Yang
dongliny@nvidia.com
英伟达公司 美国圣克拉拉

Xiaobo Zhou
w
aynexzhou@um.edu.mo
IOTSC 澳门大学 澳门特别行
政区

Dazhao Cheng*
dcheng@whu.edu.cn
武汉大学计算机学院 武汉，
中国

摘要

专家混合 (MoE) 架构通过扩大模型参数来提升模型质量。然而，在分布式训练场景中，由于显著的通信开销和专家负载不平衡，其发展受阻。现有方法仅允许通信与计算进行粗粒度重叠，虽可略微缓解通信开销，但同时显著降低了计算效率。此外，当前解决负载不平衡的方法往往会牺牲模型质量。

我们提出了 CCFuser，这是一种用于高效训练 MoE 模型的新型框架。CCFuser 用高效的跨 GPU 共享内存访问替代了 MoE 架构中常见的代价高昂的 All2All 操作。这使得可以在一个融合内核中对本地和远程数据进行并行计算，从而在 GEMM 运算中实现显著更高的计算 FLOPs。此外，CCFuser 通过一种资源高效的专家重分配策略来解决负载不均衡问题，该策略通过等价图变换在专家重分配中优化计算资源的使用，同时不牺牲统计准确率。通过整合这些优化，CCFuser 显著提升了 GPU 利用效率。在 A100 服务器上的实验

表明 CCFuser 相比当前最先进水平的方法 FastMoE 和 FasterMoE，平均分别提升 2.96 倍和 2.48 倍，最高加速比达 4.34 倍。

CCS 概念： • 计算方法论 → 并行算法。

关键词：专家混合、内核融合、GPU

美国计算机协会参考格式：

Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou 和 Dazhao Cheng. 2025年。利用跨 GPU 共享内存实现无缝的专家混合通信与计算融合。发表于 第30届 ACM 编程语言特别兴趣小组 并行编程原理与实践年度研讨会 (PPoPP'25)，2025年3月1-5日，美国内华达州拉斯维加斯。美国计算机协会，美国纽约州纽约市，13页。 <https://doi.org/10.1145/3710848.3710868>

1 引言

大型语言模型在自然语言处理等领域取得了显著突破 [1, 6] 以及计算机视觉 [2, 36]。这些成果表明，增大模型规模可以提升模型质量。然而，尽管其效果显著，训练这些大型模型会带来极高的计算成本。例如，训练 Meta 的 Llama2-70B 模型 [33] 需要在 A100-80GB GPU 上耗费 300 万 GPU 小时。

降低大规模模型计算成本的一种直接方法是采用稀疏激活 [10]。在实现稀疏激活方面已有充分证明的专家混合 (MoE) 架构，已在多项研究中被证实有效 [16, 27]。与计算所有参数的稠密模型不同，专家混合只选择性地激活一部分，从而在不成比例增加计算需求的情况下显著提升模型规模。例如，基于 MoE 的 Mixtral-8x7B 模型 [16] 包含 467 亿参数，但在计算过程中仅有 129 亿参数被激活。该模型在运行速度快六倍的同时，达到与 Llama2-70B 相当的性能。为管理

通讯作者

允许为个人或课堂使用免费制作本文全部或部分内容的数字或纸质副本，条件是不得为了盈利或商业利益制作或分发副本，并且副本在首页保留此声明及完整引用。对本文中非作者拥有的组件的版权必须予以尊重。带署名的摘要转载是被允许的。除此之外的复制、再版、发布到服务器或分发到列表，需事先获得特定许可和/或支付费用。许可请求请发送至 permissions@acm.org。PPoPP '25, 2025年3月1-5日，美国内华达州拉斯维加斯© 2025年 版权由所有者/作者保留。出版权已授权给美国计算机协会。美国计算机协会 国际标准书号 979-8-4007-1443-6/25/03...\$ 15.00<https://doi.org/10.1145/3710848.3710868>

为管理 MoE 模型的大规模参数，当前的训练系统 [14, 17, 18] 通过将专家分配到多个 GPU 以支持分布式训练，这种方法称为专家并行。尽管该方法缓解了 MoE 模型的高内存需求，但会对训练效率产生不利影响。

专家混合的专家并行依赖由英伟达 NCCL 库提供支持的 All2All 操作来进行标记的分发与收集。在这些操作期间，工作节点中的计算资源经常处于闲置状态。最近的一项研究 [14] 表明，在专家混合设置中，超过 40% 的训练时间被 All2All 通信消耗，显著影响性能。为缓解这一问题，人们提出了若干方法 [12, 19, 29, 30, 42]，通过跨算子调度来加速训练。该策略将输入数据划分为更小的批次，并设计调度算法，使通信与计算（例如 GEMM（通用矩阵乘法））重叠，从而最小化通信开销。尽管有这些进展，这些方法仍难以在通信的起始和结束阶段消除 GPU 空闲时间。此外，更精细地划分输入数据，虽然降低了通信负载，却会导致计算块变小并降低 GPU 利用率。

影响 MoE 训练效率的另一个因子是专家负载不均衡问题。在专家混合中，输入数据（标记）通过动态路由网络分发到各个专家，导致每个专家处理的标记数量不均衡，且各 GPU 之间负载失衡。策略 [10, 12, 17] 通过为每个专家可处理的标记数量设定上限来应对这一问题。然而，这些方法可能导致标记被路由到次优专家，甚至被丢弃，从而损害模型质量。其他方法 [23] 通过重新分配专家来在 GPU 之间实现负载均衡。尽管在实现均衡方面有效，但这些方法仍需要在 GPU 之间进行大量标记传输，导致过多的通信开销。此外，专家重分配的算法主要利用通信资源，导致计算资源利用不足以及 GPU 效率较差。

为了解决现有 MoE 训练框架的局限性，我们引入了 CCFuser，这是一个旨在显著提升训练效率和负载均衡的新型框架。CCFuser 具有两项关键创新：1) 一种利用跨 GPU 共享内存的 GEMM（通用矩阵乘法）内核融合，有效消除专家混合模型训练中与 All2All 操作相关的通信开销；2) 一种高效利用资源的专家重分配策略，在提升数据局部性的同时优化跨 GPU 的负载均衡，从而提高 GPU 利用率。

首先，我们不再采用传统的粗粒度算子间重叠，而是提出将通信与计算操作无缝融合到一个利用跨 GPU 共享内存的、经优化的 GEMM（通用矩阵乘法）内核中。这样的融合从策略上避免了 GPU 计算资源的

空闲，通常由独立的通信操作引起。通过在内核内同时在线程块间和线程块内两个层级上重叠通信与计算，我们有效地隐藏了访问远程数据所带来的开销，从而最大化 GPU 计算资源利用率。为实现这种无缝融合，我们将用于 All2All 操作的数据组织在跨 GPU 共享内存中，并设计了一种结合分块（tiling）方案的新数据结构。此外，通过将所有本地 GPU 专家计算整合进 GE MM（通用矩阵乘法）内核，我们显著提升了专家混合专家计算效率，并解决了 MoE 训练的主要瓶颈之一——GPU 利用率。

其次，我们通过一种资源高效的专家重新分配算法实现跨 GPU 的负载均衡。通过改造 MoE 层的结构，我们使专家分配算法与注意层计算能够同时执行，最大化通信与计算资源的利用率。根据令牌分布，我们对热门专家与冷门专家采用不同的分配策略，以提升每个 GPU 内的数据局部性。对于热门专家，我们使用专家复制以改进数据本地化。相反，对于冷门专家，我们促进跨 GPU 的专家重新分配，从而有效缓解内存压力。

总之，我们的主要贡献如下：

1. 我们提出了一种基于跨 GPU 共享内存的全新融合内核，它能够在内核内部同时处理本地和远程数据

这一创新完全消除了 All2All 操作所需的跨 GPU 同步。

2. 我们提出了一种在线程块间与线程块内同时实现通信与计算重叠的策略，有效隐藏访问远程数据的开销，并最大化 GPU 计算资源利用率。

3. 我们提出了一种资源高效的专家重分配算法，显著增强数据局部性并平衡工作负载。我们通过改造专家混合结构来降低算法开销，从而显著提升计算资源利用率。

我们将 CCFuser 集成到 PyTorch 中用于专家混合模型训练。在英伟达 A100 服务器上测试，CCFuser 相较于 FastMoE 和 FasterMoE 实现了显著的性能提升，平均加速分别为 2.96 倍和 2.48 倍，最高加速可达 4.34 倍。

2 背景与动机

2.1 专家混合结构与专家并行

门控的专家混合架构 [28] 由多个称为专家的子模型组成，通常实现为

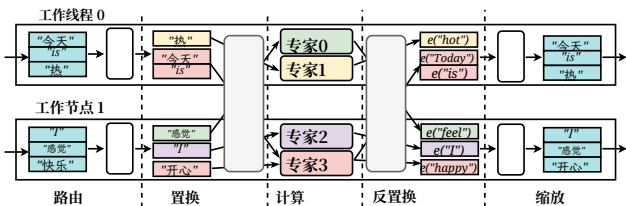


图1。一个 MoE 层以其泛化形式展示，其中 $expert_num=4$, $top_k=1$, $worker_num=2$ 。专家 0-1 位于工作节点 0，而专家 2-3 位于工作节点 1。

作为前馈网络（FFN） [10]。通过路由动态选择专家来处理输入，使模型更高效且更具表达力 [10, 27]。

在 Transformer 模型中，MoE 层通常替代 FFN 层，在不显著提高计算成本的情况下增加参数和性能。当参数超过单个 GPU 的容量时，专家并行会将 MoE 专家分布到多个 GPU 上。MoE 层的结构如图 1，包括五个阶段：路由、置换、计算、反置换和扩展性。

路由： MoE 层的第一阶段将标记分配给专家。Token 通过一个门控网络，生成标记-专家得分矩阵。经过 Softmax 归一化后，为每个标记选择前 k 个专家。

置换： 在确定了所有输入标记的专家索引后，置换阶段在每个工作节点内对标记向量进行分散与重排，使同一专家的向量得以聚集。随后通过一次 All2All 操作，将标记在各工作节点之间分发到各自对应的专家进行处理。

计算： 一旦数据完成置换，专家将在其分配的输入上执行计算以生成结果。

反置换： 在专家计算之后，通过一次 All2All 操作将结果向量重新分发回原始工作节点，随后进行本地聚合，按照其原始专家混合输入顺序对向量重新排序。

扩展性： 专家混合的最后一步根据其专家得分对输出 Token 进行缩放；对于被分配给多个专家的 Token，将其加权结果求和，以生成最终的层输出。

2.2 英伟达通信库

英伟达为多 GPU 环境提供了两种主要通信库：NCCL [24] 和 NVSHMEM [25]。

NCCL 是由英伟达开发的一款高性能通信库，专为使用多块 GPU 的深度学习和高性能计算环境而定制。它提供经过优化的集体通信操作，如全规约（All-Reduce）、广播（Broadcast）和 All-Gather。NCCL 通常与 TensorFlow 和 PyTorch 等流行的深度学习框架集成，促进跨 GPU 的高效数据同步。

相比之下，NVSHMEM 是为英伟达 GPU 设计的共享内存通信库，并已被广泛用于与图神经网络相关的研究 [37, 38]。受传统共享内存模型（SHMEM）的启发，它将 GPU 内存划分为 GPU 间共享段和私有段。该设置使得一个 GPU 上的流程可以通过 NVSHMEM 共享内存地址及其关联的 GPU ID 访问另一块 GPU 上的数据。该库提供多种用于同步和通信的 API，以满足多样化的编程模型和应用场景。不同于专注于全面通信操作的 NCCL，NVSHMEM 更具可组合性，提供更低层级的 API，使得在内核内进行更细粒度的数据访问成为可能。

2.3 动机

2.3.1 提升专家混合效率：从 NCCL 转向 NVSHMEM。

对于训练 MoE 模型，传统的专家并行执行在置换和反置换阶段都需要进行 All2All 通信，从而导致显著的通信开销。

我们在配备四块 A100 PCIe GPU 的服务器上，对一个包含 64 个专家的 MoE 层进行了实验。实验结果表明，总处理时间的 41% 被 All2All 通信所占用。这种冗长的 All2All 通信被确定为 MoE 模型训练中低效的主要来源。

All2All 通信大量占用网络资源。由于在专家混合模型中的置换、计算和反置换阶段之间存在固有的数据依赖关系，在这些通信期间 GPU 计算资源会处于空闲状态，如图 2(a)。All2All 通信持续时间越长，GPU 空闲时间越长，从而降低整体效率。现有方法

[12, 19, 21, 29, 30] 建议将输入数据划分为更小的块（小张量），并对这些块的通信和计算子任务进行策略性调度。这些策略旨在最大化计算与通信操作之间的重叠。如图 2(b)，将输入数据划分为两个块，并随后调度其子任务，可以有效地使这些块的专家计算与 All2All 通信重叠。

然而，这些方法依赖于基于 NCCL 的粗粒度通信，因而面临两个主要问题：1) 在 All2All 通信期间不能完全消除 GPU 的空闲状态，如 All2All 0-0 和 All2All 1-1 场景在图 2 (b) 中所示。2) 由于数据被划分为更小的张量，无法充分利用 GPU 的计算能力。因此，分区后的总时间开销（如 T_{e2} 在图 2 (b) 所示）比朴素的时间开销 T_{e1} 在图 2 (a) 中更长。

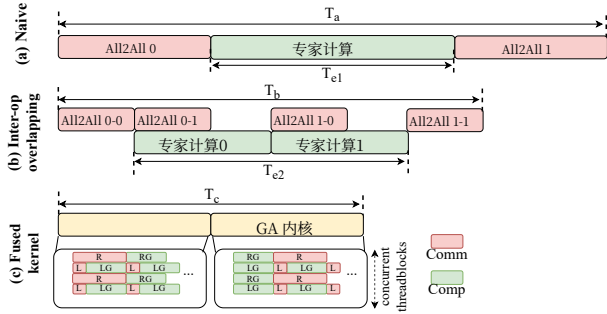


图2。专家混合的子任务调度。(a) 传统执行。(b) 以数据块调度任务。(c) 融合内核。

与 NCCL 相比, NVSHMEM 提供更细粒度的内存访问, 从而为在内核内将通信与计算无缝融合提供了机会。通过使用 NVSHMEM 替代 NCCL 来实现 All2All 功能, 可以显著减少 GPU 的空闲, 如图2 (c) 所示。然而, 采用 NVSHMEM 实现无缝的 MoE 通信-计算融合面临两个重要挑战。

(1) 复杂的数据管理。专家混合的网络路由具有动态性, 意味着专家处理的数据持续变化, 这使得融合内核从跨 GPU 共享内存中为计算准确检索正确数据的任务变得复杂。此外, 由于内核同时处理本地和远程数据, 传统 GEMM (通用矩阵乘法) 内核沿行、列维度的数据分块在频繁的远程访问下会降低性能。因此, 我们需要设计合适的数据结构和分块方案, 有效管理数据并提升融合内核的性能。

(2) 远程数据访问的高开销。尽管 NVSHMEM 相比 NCCL 提供了更细粒度的内存访问, 但它并不能从根本上降低与远程数据访问相关的开销。当线程块需要访问远程数据时, 随之而来的高通信开销可能导致操作被阻塞, 浪费计算资源并延长内核执行时间。因此, 我们的目标是充分利用因远程数据访问导致的 GPU 空闲周期来掩蔽这部分开销, 从而最大化 GPU 硬件处理单元的利用率。

2.3.2 MoE 计算中的负载均衡。 由于 MoE 模型中的动态路由, 输入标记会被分配给不同的专家进行计算。为研究负载均衡问题, 我们将 Transformer-xl model 中的 FFN 层 [5] 替换为包含 64 个专家的 MoE 层。这些专家分布在四块 A100-PCIE GPU 上, 每块 GPU 负责 16 个专家。在 enwik8 [13] 数据集上训练后, 所得指标如图 3。数据表明, 输入标记对某些专家存在显著的偏好, 导致专家之间的工作负载分配严重不均, 如所示

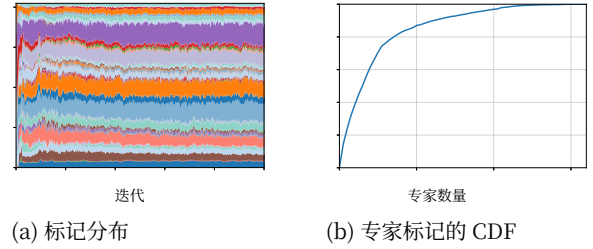


图3. MoE训练中的标记分布。

在图3(a)。这种不均衡也扩展到了 GPU 本身, 导致资源利用率不佳。

一些现有方法 [12, 17] 通过限制每个专家处理的标记数量来在 GPU 上实现负载均衡。然而, 这些方法往往会导致大量标记被重新分配给次优专家——甚至被丢弃, 最终损害模型质量。与以牺牲模型质量来实现负载均衡的以往方法不同, FlexMoE [23]通过重新分配专家, 直接在 GPU 间平衡工作负载。然而, 这种方法带来了两个不小的挑战:

(1) GPU 数据局部性差。MoE层的专家被分为热门和非热门两组。最受欢迎的前6个专家 (共64个) 处理了超过50%的输入标记, 如图3(b)。如果在分配时不考虑专家的受欢迎程度, 理想情况下, 每个 GPU 处理的本地数据占比大约为 $1/N$ (其中 N 为 GPU 数量)。这种安排会导致大量标记被传输到其他 GPU 上处理, 从而造成较差的数据局部性并降低 GPU 计算效率。

(2) GPU 计算资源的浪费。专家重分配算法在 MoE 层的门控网络与专家计算之间运行。由于来自专家计算的权重数据依赖, 专家计算被延迟, 影响训练效率。此外, 由于专家重分配算法需要在 GPU 之间交换专家权重, 它会消耗 GPU 间通信资源, 导致计算资源浪费。

3 CCFuser 设计

我们提出 CCFuser, 这是一种利用 GPU 间共享内存, 将计算与通信无缝融合以实现专家混合模型的全新方法。CCFuser 利用 NVSHMEM, 使得跨 GPU 对专家混合的输入标记的协同共享与访问成为可能。该框架在线程块间和线程块内两个层面实现通信与计算的细粒度重叠, 从而最大化 GPU 计算资源利用率。

CCFuser 引入了两项主要创新:

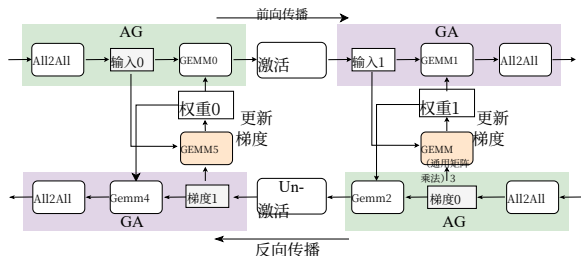


图4. 专家混合的前向传播与反向传播的数据流。

1. 基于跨 GPU 共享内存的 GEMM（通用矩阵乘法）：

该功能以跨 GPU 共享内存访问替代传统的 All2All 通信，从而实现融合的通用矩阵乘法内核。通过分层重叠通信与计算，它显著提升 MoE 计算效率。

2. **资源高效的专家重分配：**该策略根据专家在各 GPU 上的受欢迎程度进行重新分配，确保负载均衡，同时提升数据局部性，从而提高整体 GPU 通信与计算资源效率。

3.1 基于跨 GPU 共享内存的 GEMM（通用矩阵乘法）

在 MoE 模型的专家并行中，大量时间花费在置换和反置换阶段的 All2All 操作上，导致 GPU 计算资源空闲。为提高 GPU 利用率，我们设计了一种利用跨 GPU 共享内存的新颖 GEMM（通用矩阵乘法）内核。该新内核通过在线程块间和线程块内两个层面有效实现通信与计算的重叠，显著加速计算。

3.1.1 AG 和 GA 内核。考虑一个专家混合 FFN 层，其中每个专家都是一个 2 层 MLP，如图 4。在前向传播中，一次 All2All 操作会在执行第一个 MLP 的 GEMM（通用矩阵乘法）操作之前，将数据从各个工作进程传输到其对应的专家工作进程。完成第二个 MLP 的 GEMM 操作后，另一次 All2All 操作会将标记发送回其原始工作进程。在反向传播期间，该过程反向执行，并且每个 MLP 还会额外执行一次 GEMM 操作以计算权重的梯度。

传统上，All2All 操作与相邻的 GEMM 运算之间存在依赖关系，导致在 All2All 通信期间 GPU 计算资源处于空闲状态。为解决这一低效问题，我们集成了 NVSHMEM 库，将前向和反向传播中的 GEMM 运算与其对应的 All2All 操作融合为专家混合架构内的统一内核。该集成使内核能够同时处理本地和远程数据，从而消除与基于 NCCL 的 All2All 操作相关的 GPU 空闲时间。因此，我们开发了两个创新的内核：All2All-GEMM

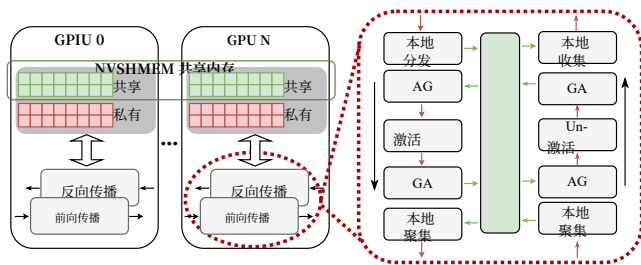


图5. 专家混合操作与跨 GPU 共享内存之间的交互。红色方框表示在前向传播和反向传播期间的交互。箭头显示数据流动。

（AG）内核，它在前向和反向传播期间同时处理 All2All 和第一个 GEMM 运算；以及 GEMM-All2All（GA）内核，负责第二个 GEMM 和 All2All 操作。这种在单一内核中混合本地与远程工作负载的策略显著提升 GPU 利用率，同时优化计算流程和资源效率。

如何使用 NVSHMEM 在计算中替换 All2All 操作？在专家混合架构中，All2All 操作在第一个 MLP 之前和第二个 MLP 之后都是必不可少的。为了用 NVSHMEM 替代基于 NCCL 的 All2All，我们利用 NVSHMEM 共享内存高效地存储第一个 MLP 的输入和第二个 MLP 的输出。为此，我们将全局 GPU 内存划分为跨 GPU 共享内存和私有内存。跨 GPU 共享内存存放专家混合的输入和输出，使所有 GPU 都可以访问它们，而私有内存用于存放权重、激活以及其他关键张量。

该设置促进了专家混合操作与跨 GPU 共享内存之间的交互，如图 5。在执行 AG 操作之前，我们对 *local scatter* 的结果执行写操作到跨 GPU 共享内存。在 AG 内核期间，NVSHMEM 远程数据访问 API *nvsh-memx_getSIZE_nbi_block* 获取用于 GEMM（通用矩阵乘法）计算的 MoE 层的输入数据。随后，在 GA 内核中，完成 GEMM（通用矩阵乘法）计算后，使用 *nvsh-memx_putSIZE_nbi_block*。最后，每个 GPU 使用 *localgather* 操作，根据 MoE 输入对来自本地跨 GPU 共享内存的数据重新排序。反向传播与前向传播镜像。

通过采用基于跨 GPU 共享内存的融合内核，我们完全消除了其他方法中 All2All 操作通常需要的跨 GPU 同步操作，从而显著提升 GPU 利用率。

3.1.2 数据格式与数据铺片。为高效将数据映射到 GPU 的张量核心单元，我们沿行维度将数据划分为块，并将这些块的源数据存储于 NVSHMEM 共享内存中。当内核

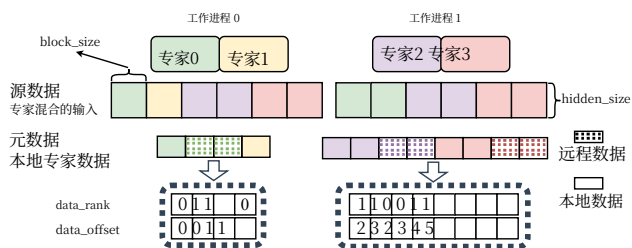


图 6。专家混合的源数据和元数据。

启动时，每个线程块会确定其需要访问的 NVSHMEM 共享内存地址。如图 6，每个工作线程会存储与本地专家所需数据相关的元数据。在 2 层 MLP 的专家计算中，输入和输出矩阵的行数及序列是固定的。因此，这些元数据可以一致地由前向传播和反向传播 AG 内核以及 GA 内核共享。

元数据变量 `data_rank` 和 `data_offset` 分别表示 GPU 索引和 NVSHMEM 共享内存中的偏移量。这些变量对于 AG 和 GA 内核在共享内存中准确且高效地定位专家数据至关重要。鉴于我们只需存储一组大小为 $block_size \times hidden_size$ 的元数据，这部分额外元数据的内存需求极小，从而优化系统资源使用。

如何在线程块之间实现高效的数据切片？图 7(a) 展示了用于 GEMM（通用矩阵乘法）计算的常见数据切片方法，即沿行和列两个维度将输出矩阵划分为子矩阵，并将每个子矩阵分配给一个线程块进行计算。然而，这种传统的数据切片方法不适用于 AG 和 GA 内核，原因在于远程数据访问的低效。在这些内核中，访问远程 GPU 上的数据可能比访问本地 GPU 内存慢数十到数百倍。如图 7(a) 所示，在计算阴影部分时，可能有四个线程块需要访问相同的远程数据，导致内核性能下降。

对于我们的 AG 和 GA 内核，我们设计了一种数据分块策略，仅沿行维度划分输出矩阵，如图 7 (b)。该方法确保在计算过程中所有远程数据只被访问一次，显著减少低带宽数据访问次数，并提升整体 GPU 利用率。

借助新的数据分块，远程和本地数据块可以在内核中分配给不同的线程块，从而使 GPU 的 SM 能够同时处理这两类数据。这些线程块的调度由 GPU 调度器管理，以优化资源利用：当一个线程块正在访问远程数据时，其他线程块可以并行计算本地数据。这种在线程块之间

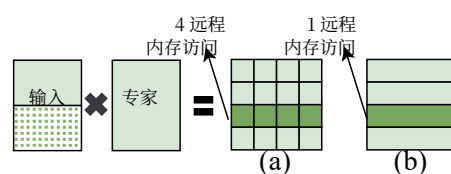


图 7。用于稀疏 GEMM（通用矩阵乘法）的数据分块，按 (a) 行和列维度划分，(b) 行维度划分。

跨不同 SM 的通信与计算重叠的策略，显著提升了整体效率。

3.1.3 细粒度线程块内重叠。 如前所述，线程块间重叠可以提升 GPU 占用率。但不能最大化 GPU 的计算资源。当线程块访问远程数据块时，线程在等待数据期间常常会遭遇显著延迟，从而延长整体内核运行时间。

为了解决这一问题，我们让线程块同时处理远程和本地数据，使通信延迟得以重叠，从而提升 GPU 利用率。

如图 8 所示，左侧展示了在线程块内对数据块的顺序处理。它表明，当访问远程线程块 1 和线程块 2 时，线程块必须等待远程数据访问完成，导致处于阻塞状态。为尽量减少线程块因远程数据访问而被阻塞的时间，我们提出了一种在线程块内进行通信与计算自适应重叠的方法，如图 8 的右侧所示。在 AG 内核中，我们首先使用异步读取 API 访问远程数据块，随后进行本地数据计算，然后再对远程数据块进行计算。相反，在 GA 内核中，先对远程数据块进行计算，随后将结果以异步方式写回，最后进行本地数据计算。该方法使 SM 能够在进行本地数据计算的同时并发处理远程数据的读取和写操作，从而最大化 GPU 计算资源利用率。

由于 NVSHMEM 的远程数据访问基于点对点（P2P）模型，其速度取决于涉及的两块 GPU 之间的带宽速率。如图 8 所示，线程块 1 通过 NVLINK 互连，而线程块 2 通过 PCIe 桥连接。因此，线程块 1 相较于线程块 2。因此，必须根据远程与本地 GPU 之间的带宽连接，选择适当数量的本地数据块与远程数据块进行重叠。例如，如图 8 所示，我们将一个本地数据块与远程线程块 1 重叠，并将两个本地数据块与远程线程块 2 重叠，以高效地平衡数据处理速度。

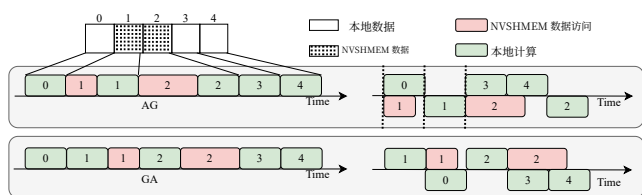


图8。在线程块内，通信与计算之间的自适应重叠。

3.1.4 多专家融合。 在默认的MoE层中，每个专家都会独立处理分配到的输入标记。然而，启动多个内核会引入额外的系统开销。此外，Token 分布不均会导致低效，因为未充分利用的专家处理的Token 过少，无法充分利用GPU。将多个专家内核横向融合为单个内核可以同时解决这两个问题。

在融合多个专家内核时，AG 和 GA 内核中的每个线程块都需要获取所需的专家权重以处理输入标记。有必要引入元数据，即，`data_expert_id` 以帮助定位每个数据块。例如，worker 0 的 `data_expert_id` 可能是 0,0,0,1。在内核中，线程块根据元数据的 `data_rank` 和 `data_offset` 访问本地或远程数据，使用 `data_expert_id` 定位本地专家的权重，以便于融合内核计算。

在专家混合的反向传播中，为计算权重梯度，将多个专家内核进行融合会把计算从二维问题转化为三维问题。在这一计算过程中管理数据块存在挑战。为此，我们提出沿三维对输出梯度进行分块：专家数量，以及权重矩阵的行与列。这种数据分块策略可确保更强的并行性。此外，不同于在数据和权重上执行 GEMM（通用矩阵乘法）的 AG 和 GA，用于梯度计算的 GEMM（通用矩阵乘法）以前向传播 激活 以及反向传播梯度 作为输入。因此，需要一个额外的元数据，`expert_data_offset`，用于索引正确的 激活 和 梯度 用于融合的通用矩阵乘法内核的线程块。如图 6 所示，worker 0 的 `expert_data_offset` 为 {0,3,4}。这意味着第一个专家的数据跨越 块 0-2，第二个专家的数据是 块 3。

通过水平融合，MoE层中的所有专家在 MoE 前向和反向传播中被融合到单一内核中，大幅加速计算。

3.2 资源高效的专家重分配

专家混合中的专家对标记的分布往往不均衡，导致大量标记被发送到其他GPU进行计算。随着发送的标记数量增加，通信开销上升，进而造成GPU 利用率较低、效率下降。我们的专家重新分配方法通过增强

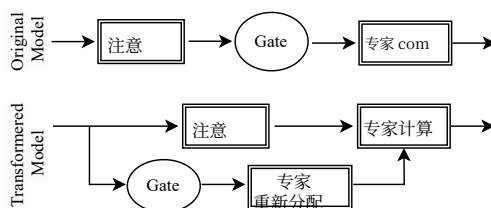


图9。MoE层的变换。

基于标记分布的数据局部性。具体而言，我们修改 MoE 层并预运行门控网络以确定标记分布。随后，我们在GPU之间重新分配专家，以提升数据局部性，在不牺牲统计准确率的前提下确保GPU间的工作负载均衡，并将专家重新分配与不含数据依赖的操作并行化，以优化GPU 通信和计算资源效率。

3.2.1 MoE层的变换。 如果在 MoE 计算之前执行专家重分配算法，MoE 计算就必须等待重分配算法完成，从而降低训练效率。为了解决这一问题，我们通过同时执行门控网络和注意的计算来改变MoE层的结构，如图 9。注意模块的输入也作为门控网络的输入，使我们能够为每个标记选择合适的专家。预先确定令牌路由信息后，我们可以提前执行专家重分配算法。需要注意的是，专家重分配算法与注意机制之间不存在数据依赖。此外，专家重分配主要消耗GPU间带宽，而注意计算利用GPU的计算资源。因此，专家重分配算法与注意层计算并行处理。

对专家混合的这种变换是合理的，因为门控网络通常采用一种简单的模型架构，在该架构中，路由或专家选择往往由一些直观的特征决定，例如 Token 的句法角色（如名词、动词）或语义含义（如时间、数字）[18]。这些特征对于每个标记都是固定的。此外，由于门控网络的简单性，即便在训练期间使用来自上一层的特征向量，它也能捕获标记的句法或语义属性。基于上述原因，这种对专家混合的变换是有效的，我们已在第 4.5 节中验证了它不会影响模型的收敛速度。

3.2.2 专家重分配。 基于不同GPU上的Token分布，我们可以实现我们的专家重分配算法，以增强GPU上的数据本地化。该算法的结构如下：在MoE层中，给定不同GPU上的Token分布 N （总核心数），我们需要将集合 E （本地专家数）中的所有专家分配到多GPU

平台 G，考虑GPU之间的带宽 B（批次大小），以确定最终的专家分配方案 P（处理器）。我们的目标函数是在GPU上最大化数据本地化，计算为：

$$\mathcal{L}(g', \mathcal{P}) = \frac{\sum_{e \in \mathcal{E}}^{(e, g') \in \mathcal{P}} \mathcal{N}_{e, g'}}{\sum_{e \in \mathcal{E}} \mathcal{N}_{e, g'}} \quad (1)$$

专家混合中的专家被分为热门专家和冷门专家，对每一类需要采用不同的分配策略。对于热门专家，由于它们处理了很大比例的输入数据标记，将其计算保持在本地更高效。对于冷门专家，由于处理的标记较少，我们将其分布在多个GPU上以缓解内存压力。

我们通过分析该GPU上的专家的标记分布 $\mathcal{N}_{e, g'}$ 来识别GPU g' 上的热门专家。因此，我们将 $\mathcal{N}_{e, g'}$ 按降序排序，以选出满足以下条件的该GPU上热门专家的最小集合E（本地专家数） p 。

$$\mathcal{E}_p = \arg \min_{\mathcal{E}_p} \{ |\mathcal{E}_p| \mid \frac{\sum_{e \in \mathcal{E}_p} \mathcal{N}_{e, g'}}{\sum_{e \in \mathcal{E}} \mathcal{N}_{e, g'}} > 50\% \} \quad (2)$$

其中 $|\mathcal{E}|$ （本地专家数）表示集合E（本地专家数）的元素个数。我们要求热门专家的标记总数占本地标记的比例超过50%，因为至少需要50%的本地数据来解决重叠。得到E（本地专家数） p 后，我们将元组 $\{e', g'\}$ 添加到专家重分配计划P（处理器），其中 $e' \in \mathcal{E}$ （本地专家数） p 。

对于冷门专家，我们需要将其分配到不同的 GPU。由于内核实现基于 NVSHMEM 并遵循 P2P 通信模型，其访问速度与 GPU 之间的通信带宽直接相关。因此，在分配冷门专家时，我们需要考虑 GPU 之间的网络拓扑，最大化带宽资源利用率。对于特定专家 e' 的分配，我们应遵循以下标准选择合适的 GPU g' 。

$$g' = \arg \max_{g'} \left\{ \sum_{g \in \mathcal{G}} \mathcal{N}_{e', g} \times \mathcal{B}_{g, g'} \right\}, \text{ where } g' \in \mathcal{G} \quad (3)$$

然后，将元组 $\{e', g'\}$ 添加到分配方案 P（处理器）。

在专家重分配算法中，我们首先计算上一份分配方案 P（处理器）使用公式1。如果 P（处理器）满足最小数据局部性阈值 l_b ，则予以保留。否则，我们生成一个新方案，P（处理器）'。随后，依据公式 2，并将其纳入 P（处理器）' 其 N（总核心数） e, g 值设为 0，以避免后续步骤中的冲突。接着，我们根据公式 3 将冷门专家分配到合适的 GPU，并将其添加到 P（处理器）'。最后，按照 P（处理器）'，完成所有专家的分配，得到更新后的分配方案。

借助专家分配算法，我们可以显著提升GPU上的数据局部性。这解决了负载不均衡问题，并提高了内核计算效率。

由于模型训练具有迭代性质，每个GPU上的已分配专家基本保持不变，因而偶尔的专家重新分配仅带来极小的开销。因此，我们的方法在训练期间引入的额外开销极小。

3.3 实现

基于 CUDA 11.7 和 PyTorch 2.0.1 构建，CCFuser 通过 PyTorch Extensions 无缝集成到 PyTorch 生态中。为优化性能，CCFuser 采用专家并行，将专家分布到多块 GPU 上，同时对其他模型组件应用数据并行。它提供易用的 Python API，使开发者能够高效实现自定义 MoE 模型。CCFuser 使用 PyTorch（v2.0.1）、CUDA（v11.7）、OpenMPI（v4.1.1）、NVSHMEM（v2.9.0）和 cuDNN（v8.2）进行编译与链接。开源成果已在 GitHub 上提供：

<https://github.com/WongHuLin/CCFuser.git>。

4 评估

4.1 实验设置

测试平台。我们在两台服务器上评估 CCFuser。第一台服务器，*pinnacle*，配备两颗 Intel 8269CY 处理器（每颗 26 核心、52 线程、2.50 GHz）。其包括 256 GB 系统内存和四块 英伟达 A100 PCIe GPU（每块 40 GB 内存），通过 NVLink 桥和 PCIe 总线连接。该服务器代表了典型的深度学习训练工作站。第二台服务器，*vertex*，配备两颗 AMD EPYC 7543 处理器（每颗 32 核心，2.8GHz）以及四块 英伟达® Tesla A100 GPU（每块 40 GB 内存）。这些 GPU 通过 NVLink 以全互连拓扑互连，提供高达 600 GB/s 的 GPU 间通信带宽。该配置非常适合高性能计算。

方法。我们将 CCFuser 与两个用于 MoE训练优化的框架进行比较。FastMoE [11] 是一个基于 PyTorch 和通用加速器的 MoE训练系统，而 FasterMoE [12] 在 Fast MoE 之上通过调度通信和计算操作，以在 MoE训练期间最小化通信开销。

指标。我们使用三个关键指标评估 CCFuser：

- MoE层时间：一个 MoE层的计算与通信时间。
- 训练步数时间：完成一次训练步数。
- 内核执行时间：完成融合内核（AG 或 GA）的执行。

为收集这些指标，我们使用 CUDA Events 来测量 CUDA 内核和通信的执行时间。结果在经过 20 次预热运行后，对随后 80 个步数取平均。

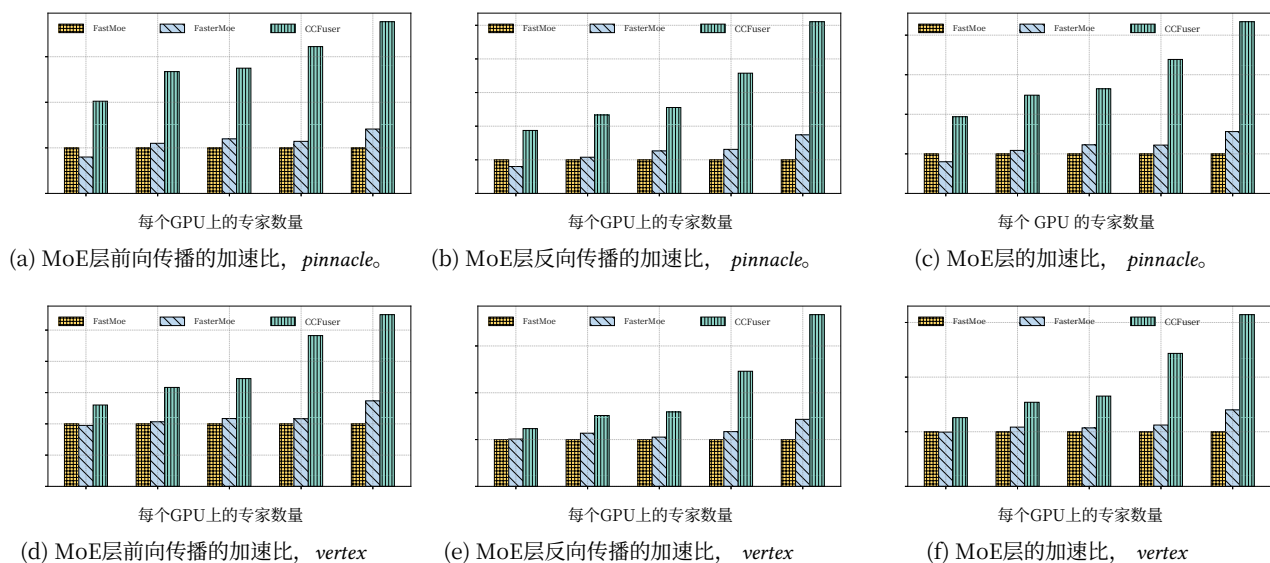


图10. 不同专家数量下的MoE层相对于FastMoE的训练时间加速比。

4.2 MoE层的性能

我们首先在两个平台上评估 MoE层 的性能。评估方法是在所有框架中使用相同的输入配置（批大小 = 16，序列长度 = 512，top_k = 2，模型维度 = 1024），并在实验过程中增加每块 GPU 上的专家数量，测量专家混合的前向传播、反向传播和总时间。将 FastMoE 的性能作为结果的基线。

图10展示了在不同配置下MoE层的训练时间加速情况。我们特别关注CCFuser在MoE层的前向传播和反向传播中提供的性能提升。如图 10(a), 10(b), 10(d) 和 10(e)所示，CCFuser的性能提升随专家数量增加而增大，相对于FastMoE，前向传播平均提升2.42x，反向传播平均提升2.59x。与前向传播相比，MoE层的反向传播包含两个用于计算权重梯度的额外GEMM运算，CCFuser也对其进行了优化。因此，反向传播的性能提升更为显著。

关于MoE层的总体时间，如图 10(c) 和10(f)所示，CCFuser优于FastMoE和FasterMoE，平均性能提升达到2.96x和2.48x，在 *pinnacle*，以及2.0x和1.73x，在 *vertex*，分别对应两种基线。相较而言，在 *pinnacle* 相较于 *vertex*，主要原因在于 *vertex* GPU通过NVLink互连，提供了更高的通信带宽。这降低了MoE层的通信开销，并限制了我们的CCFuser方法的优化潜力。

与 FastMoE 相比，FasterMoE 可以对专家的计算与通信操作进行重叠，从而

表1. 模型的配置。

模型	批次 size	Seq len	模型 dim	MoE 块	总计 块	专家 num
M-GPT	8	1024	768	1	12	64
M-BERT	32	512	768	4	12	64
M-Trans-xl	16	512	512	12	12	64

隐藏通信开销。然而，其效率随专家数量而变化。当专家数量较少时，FasterMoE 甚至可能表现不如 FastMoE，如图 10(c) 和图10(f)，其中专家数量为 4。相比之下，我们的实现将专家混合的通信与计算无缝融合进 CUDA 内核，消除由通信导致的 GPU 空闲时间。这在我们的实验中带来了显著的加速，尤其是在专家数量增加时，通信开销更为显著，从而带来更大的优化收益。

4.3 专家混合模型的端到端性能

我们使用三种模型评估了 CCFuser：MoE-GPT、MoE-BERT 和 MoE-Transformer-xl。BERT 是基于 Transformer 架构编码器的代表性模型，而 MoE-GPT 和 MoE-Transformer-xl 是基于 Transformer 架构解码器的代表性模型。这些模型的配置如表 1所示。所有这些模型都由 12 个 Transformer 块组成。对于 BERT 和 GPT 模型，我们将部分块替换为 MoE 块：具体而言，在 BERT 模型中替换第 2、5、8 和 11 个块，在 GPT 模型中替换第 11 个块。在 Transformer-xl 模型中，所有块都替换为 MoE 块。

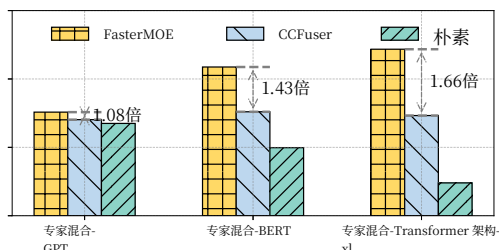


图11. CCFuser、Naive 和 FasterMoE 的性能。

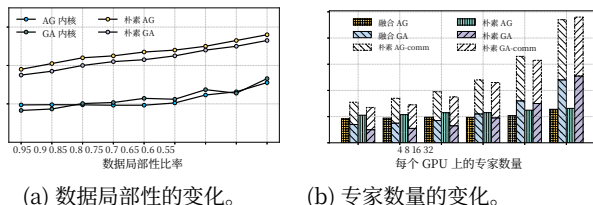


图12. 融合内核对性能的影响。

我们在三种模型上比较了 FasterMoE、Naive（不含 MoE 层）和 CCFuser 的性能，并以训练步骤时间作为系统效率的指标。图 11 显示了 FasterMoE 与 CCFuser 的性能对比。将原始模型的 FFN 层替换为 MoE 层会引入额外的通信和计算开销，使得 FasterMoE 和 CCFuser 相较于 Naive 的步骤时间更高。然而，我们的方法显著降低了这部分开销，相较于 FasterMoE 在三种模型上分别实现了 1.08x、1.43x 和 1.66x 的加速比。尽管如此，与 MoE 层中的加速比相比，端到端的整体性能提升略逊一筹。这是因为每次迭代的时间开销不仅包括 MoE 层的通信和计算，还包括模型的其他组件，例如注意模块和嵌入层。由于我们的方法未对这些组件进行优化，整体性能提升受到影响。

4.4 融合内核的分析

本论文的关键贡献是将专家混合通信和计算融合到 CUDA 内核中。为了评估融合内核的性能，我们从两个角度对其进行测试：数据局部性和专家数量。

数据局部性是指融合内核在 GPU 上进行计算时需要访问的远程数据量。图 12(a) 展示了在不同数据局部性下，与朴素实现（其中通信与计算相互独立）相比，融合内核的性能变化。随着数据局部性下降，朴素实现的耗时线性增长，主要原因是由于较差的数据局部性带来的通信开销增加，如图 12(a) 所示。然而，融合内核保持了相对稳定的耗时

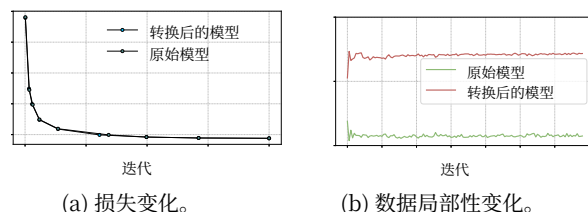


图13. 资源高效专家重分配的影响。

当数据局部性高于 0.7 时，该开销保持相对稳定。这种稳定性归功于我们内核中的线程块间与线程块内的重叠机制，它们能够有效隐藏访问远程数据的通信开销。尽管如此，随着数据局部性进一步降低，融合内核的性能也会逐渐下降。

图 12(b) 展示了通过改变专家数量来评估融合内核性能的结果。随着 MoE 层中的专家数量增加，朴素实现的通信和计算开销都会上升。在输入数据大小保持不变的情况下，时间开销的增加主要是因为每个专家都需要分别进行通信和计算。这会导致多次启动通信和计算操作，且每次操作处理的数据量更小，从而降低效率。相比之下，融合内核保持了稳定的性能，因为它通过重叠机制隐藏通信开销，并在融合内核内合并所有专家的计算，从而降低计算开销。这确保即使专家数量发生变化，融合内核仍然高效。

4.5 专家重新分配的分析

nt

我们研究转换后的 MoE 层的有效性。我们将第 3.2.1 节中讨论的变换应用于 MoE-Transformer-xl 模型中的所有 MoE 层，并使用 enwik8 数据集 [13] 在 *pinnacle* 服务器。图 13(a) 展示了两个模型在整个训练过程中的损失值。可以看出，转换后模型与原始模型在训练期间的损失值几乎相同。这表明，转换后模型的门控网络仍能有效捕获标记路由信息，从而不影响模型的收敛。

图 13(b) 展示了在训练期间实施资源高效的专家重新分配后数据局部性的变化。起初，由于热门专家的不稳定性，数据局部性显著波动。然而，随着训练的推进，资源高效的专家重新分配算法显著提升了本地 GPU 上的数据局部性，将其稳定在约 70%，而未进行专家重新分配的实现则不足 20%。这种改进

为优化融合内核创造了有利条件，从而提升训练效率。

5 相关工作

随着稀疏激活的 MoE 模型在 Transformer 架构中的成功应用 [9, 16, 17, 28], 研究越来越关注通过算法和系统优化来提升 MoE 训练性能。

算法优化：当前针对 MoE 模型的工作主要解决与负载不平衡相关的性能问题。GShard [17] 以及其他研究 [12, 44] 引入了专家容量的概念，通过限制每个专家可处理的令牌数量来平衡工作负载。这些方法可能导致令牌丢弃或次优的专家分配，最终损害模型性能。CCFuser 通过重新分配专家来平衡 GPU 负载，而不改变令牌路由过程，从而提升训练效率。

系统优化：分布式混合专家训练受到 All2All 通信的制约。因此，一些优化方法从两个方面着手提升 All2All 性能：优化 All2All 原语以利用更高的带宽 [14, 35, 39], 以及压缩待传输的数据以减少通信时间 [43]。此外，其他方法 [4, 12, 19, 21, 29, 30, 41] 旨在通过与其他子任务重叠来隐藏 All2All 通信的开销，从而提升训练效率。相比之下，CCFuser 通过在内核内并发处理本地和远程数据，并通过线程块间与线程块内的重叠来隐藏远程数据访问延迟，从而消除对 All2All 通信的依赖。

通信调度技术已被广泛用于加速分布式训练 [3, 15, 31, 34, 37]。Centauri [3] 通过沿多个维度划分通信，优化了分布式训练中的集体通信，例如 AllReduce 和 AllGather（全收集）。它从三个角度对任务进行调度：算子、层和模型，从而实现通信延迟的重叠并提升硬件利用率。此外，诸多方法 [7, 8, 14, 20, 22, 26, 32, 40] 采用了不同的并行策略，包括专家并行、流水线并行和模型并行，以加速在大规模 GPU 上的分布式训练。这些系统优化与我们的工作正交的，并且可以与 CCFuser 集成，以进一步提升训练效率。

6 结论

CCFuser 是一个新颖的框架，通过集成 GPU 间共享内存技术加速专家混合模型的训练。它将通信和计算融合到 CUDA 内核中，消除现有框架中常见的跨 GPU 同步开销。它在内核内采用线程块间和线程块内的重叠，以隐藏远程数据

访问开销并提升 GPU 利用率。CCFuser 还引入了一种资源高效的专家重分配策略，改善 GPU 数据局部性和工作负载均衡，从而降低与专家重分配相关的开销并提升训练性能。实验结果表明，CCFuser 显著加速了 MoE 训练。

致谢

本工作得到了中国国家自然科学基金（62341410）、中国国家重点研发计划（2023YFE0205700）以及澳门特别行政区科学技术发展基金（FDCT）项目（0078/2023/AMJ）的资助。

参考文献

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever 和 Dario Amodei. 语言模型是小样本学习者。载于 神经信息处理系统进展 (*NeurIPS*), 第1877–1901页, 2020年–。[2] Junbum Cha, Wooyoung Kang, Jonghwan Mun 和 Byungseok Roh. Honeybee: 用于多模态大型语言模型的局部性增强型投影器。载于 *IEEE/CVF 计算机视觉与模式识别会议 (CVPR)* 论文集, 第13817–13827页, 2024年。[3] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang 和 Chao Yang. Centauri: 通过通信划分, 为大规模模型训练中的通信-计算重叠实现高效调度。载于 第29届ACM 编程语言与操作系统体系结构支持国际会议(*ASPLOS*) 论文集, 第178–191页, 2024年。[4] Zihao Chen, Chen Xu, Weining Qian 和 Aoying Zhou. 用于高效流水线 DNN 训练的弹性平均。载于 第28届ACM SIGPLAN 并行程序设计原理与实践年度研讨会 (*PPoPP*) 论文集, 第380–391页, 2023年。[5] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le 和 Ruslan Salakhutdinov. Transformer-XL: 超越固定长度上下文的注意力语言模型。载于 第57届计算语言学协会年会 (*ACL*) 论文集, 第2978–2988页, 2019年。[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee 和 Kristina Toutanova. BERT: 用于语言理解的深度双向 Transformer 的预训练。载于 2019年计算语言学协会北美分会会议 (*ACL*) 论文集, 第4171–4186页, 2019年。[7] Jiangsu Du, Jinhui Wei, Jiazhi Jiang, Shenggan Cheng, Dan Huang, Zhiguang Chen 和 Yutong Lu. Liger: 在分布式大规模模型推理中交错使用算子内与算子间并行。载于第29届ACM SIGPLAN 并行程序设计原理与实践年度研讨会 (*PPoPP*) 论文集, 第42–54页, 2024年。[8] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia 等。Dapple: 一种用于训练大规模模型的流水线数据并行方法。载于第26届ACM SIGPLAN 并行程序设计的原理与实践研讨会 (*PPoPP*) 论文集, 第431–445页, 2021年。

- [9] Jiarui Fang, Yang Yu, Chengduo Zhao 和 Jie Zhou. Turbotransformers: 一种用于 Transformer 模型的高效 GPU 服务系统。在

- Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 389–402, 2021.
- [10] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research (JMLR)*, pages 1–39, 2022.
 - [11] Jiaao He, Jiezhong Qiu, Aohan Zeng, Zhilin Yang, Jidong Zhai, and Jie Tang. Fastmoe: A fast mixture-of-expert training system. *arXiv preprint arXiv:2103.13262*, 2021.
 - [12] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 120–134, 2022.
 - [13] Marcus Hutter. the human knowledge compression contest. <http://prize.hutter1.net/>, 2012.
 - [14] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. Tutel: Adaptive mixture-of-experts at scale. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
 - [15] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 402–416, 2022.
 - [16] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gerv  t, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mixtral of experts. *CoRR*, 2024.
 - [17] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. {GS}hard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
 - [18] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning (ICML)*, pages 6265–6274, 2021.
 - [19] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed {MoE} training and inference with lina. In *2023 USENIX Annual Technical Conference (ATC)*, pages 945–959, 2023.
 - [20] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 663–679, 2023.
 - [21] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference (SIGCOMM)*, pages 486–498, 2023.
 - [22] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–15, 2021.
 - [23] Xiaonan Nie, Xupeng Miao, Zilong Wang, Zichao Yang, Jilong Xue, Lingxiao Ma, Gang Cao, and Bin Cui. Flexmoe: Scaling large-scale sparse pre-trained model training via dynamic device placement. In *Proceedings of the ACM on Management of Data (PACMOD)*, pages 1–19, 2023.
 - [24] Nvidia. Nvidia collective communication library (nccl). <https://developer.nvidia.com/nccl>, 2024.
 - [25] Nvidia. Nvshmem communication library. <https://developer.nvidia.com/nvshmem>, 2024.
 - [26] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (SIGKDD)*, pages 3505–3506, 2020.
 - [27] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, Andr   Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8583–8595, 2021.
 - [28] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
 - [29] Shaohuai Shi, Xinglin Pan, Xiaowen Chu, and Bo Li. Pipemoe: Accelerating mixture-of-experts through adaptive pipelining. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications (INFOCOM)*, pages 1–10, 2023.
 - [30] Shaohuai Shi, Xinglin Pan, Qiang Wang, Chengjian Liu, Xiaozhe Ren, Zhongzhe Hu, Yu Yang, Bo Li, and Xiaowen Chu. Schemoe: An extensible mixture-of-experts distributed training system with tasks scheduling. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, pages 236–249, 2024.
 - [31] Jaeyong Song, Jinkyu Yim, Jaewon Jung, Hongsun Jang, Hyung-Jin Kim, Youngsok Kim, and Jinho Lee. Optimus-cc: Efficient large nlp model training with 3d parallelism aware communication compression. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 560–573, 2023.
 - [32] Zhenbo Sun, Huanqi Cao, Yuanwei Wang, Guanyu Feng, Shengqi Chen, Haojie Wang, and Wenguang Chen. Adapipe: Optimizing pipeline parallelism with adaptive recomputation and partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 86–100, 2024.
 - [33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aur  lien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, 2023.
 - [34] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of*

- the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), page 93–106, 2022.
- [35] Weihua Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Accelerating distributed dlm training with optimized tt decomposition and micro-batching. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2024.
 - [36] Wenhui Wang, Jifeng Dai, Zhe Chen, Zhenhang Huang, Zhiqi Li, Xizhou Zhu, Xiaoqi Hu, Tong Lu, Lewei Lu, Hongsheng Li, Xiaogang Wang, and Yu Qiao. Internimage: Exploring large-scale vision foundation models with deformable convolutions. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14408–14419, 2023.
 - [37] Yuke Wang, Boyuan Feng, Zheng Wang, Tong Geng, Kevin Barker, Ang Li, and Yufei Ding. MGG: Accelerating graph neural networks with Fine-Grained Intra-Kernel Communication-Computation pipelining on Multi-GPU platforms. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 779–795, 2023.
 - [38] Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Scaling new heights: Transformative cross-gpu sampling for training billion-edge graphs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2024.
 - [39] Yaqi Xia, Zheng Zhang, Hulin Wang, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Redundancy-free high-performance dynamic gnn training with hierarchical pipeline parallelism. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, page 17–30, 2023.
 - [40] Tailing Yuan, Yuliang Liu, Xucheng Ye, Shenglong Zhang, Jianchao Tan, Bin Chen, Chengru Song, and Di Zhang. Accelerating the training of large language models using efficient activation rematerialization and optimal hybrid parallelism. In *2024 USENIX Annual Technical Conference (ATC)*, pages 545–561, 2024.
 - [41] Mingshu Zhai, Jiaao He, Zixuan Ma, Zan Zong, Runqing Zhang, and Jidong Zhai. {SmartMoE}: Efficiently training {Sparsely-Activated} models through combining offline and online parallelization. In *2023 USENIX Annual Technical Conference (ATC)*, pages 961–975, 2023.
 - [42] Zheng Zhang, Donglin Yang, Yaqi Xia, Liang Ding, Dacheng Tao, Xiaobo Zhou, and Dazhao Cheng. Mpipemoe: Memory efficient moe for pre-trained models with adaptive pipeline parallelism. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 167–177, 2023.
 - [43] Qinghua Zhou, Pouya Kousha, Quentin Anthony, Kawthar Shafie Khorrassani, Aamir Shafi, Hari Subramoni, and Dhabaleswar K Panda. Accelerating mpi all-to-all communication with online compression on modern gpu clusters. In *International Conference on High Performance Computing*, pages 3–25, 2022.
 - [44] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 7103–7114, 2022.